

Narrowing and Discriminated Unions

JavaScript & TypeScript Essentials

Cloud Contraptions LLC - www.cloudcontraptions.com

Reserve path. Runs only if the class finishes 01-30. Module 12's exit ticket asks how you safely use a `string | number`, which *is* narrowing.

A Union Is Only Useful Once Narrowed

```
function format(id: number | string) {  
  id.toUpperCase();  
  // Error: 'toUpperCase' does not exist on type 'number'  
}
```

- TypeScript only lets you use what **every** member of the union has
- **Narrowing** is proving which member you actually hold
- Inside the check, TS refines the type for you, no casting

typeof Narrows Primitives

```
function format(id: number | string): string {  
  if (typeof id === "string") {  
    return id.toUpperCase(); // narrowed to string  
  }  
  return id.toFixed(0);      // narrowed to number  
}
```

- Works for "string", "number", "boolean", "function", "object", "undefined", "bigint", "symbol"
- Remember `typeof null === "object"`; it will not narrow away `null`

instanceof Narrows Class Instances

```
function describe(x: unknown): string {  
  if (x instanceof Date) return `date: ${x.getFullYear()}`;   
  if (x instanceof Error) return `error: ${x.message}`;   
  return `other: ${String(x)}`;  
}
```

- The parameter is `unknown`, not `any`; TS refuses to touch it until you have proved something
- This is exactly why `unknown` is the safe choice for external data

in Narrows Object Shapes

```
type Circle = { radius: number };  
type Square = { side: number };  
  
function area(shape: Circle | Square): number {  
  if ("radius" in shape) return Math.PI * shape.radius ** 2;  
  return shape.side ** 2;  
}
```

- Distinguishes shapes by which property exists
- Fine for two members; fragile once the union grows

Truthiness and Equality Narrowing

```
function greet(name?: string): string {  
  if (!name) return "Hello, stranger";  
  return `Hello, ${name.trim()}`; // definitely a string here  
}  
  
function check(x: string | null) {  
  if (x === null) return "empty";  
  return x.length; // null is gone  
}
```

- An early `return` narrows the **rest of the function**
- This is why guard clauses and TypeScript get along so well

Discriminated Unions

- Give every member a shared property whose type is a **literal**
- `switch` on that tag and each branch narrows to exactly one member

```
type Shape =  
  | { kind: "circle"; radius: number }  
  | { kind: "square"; side: number }  
  | { kind: "rect"; width: number; height: number };  
  
function area(s: Shape): number {  
  switch (s.kind) {  
    case "circle": return Math.PI * s.radius ** 2;  
    case "square": return s.side ** 2;  
    case "rect":   return s.width * s.height;  
  }  
}
```

Why the Tag Beats `in`

```
// With `in`, adding a member means auditing every check by hand
if ("radius" in shape) { /* ... */ }

// With a tag, the compiler tracks it for you
switch (shape.kind) { /* ... */ }
```

- One property to read, whatever the union's size
- Reliable even when two members share property names
- The standard way to model events, actions, and API results

Exhaustiveness With `never`

```
function assertNever(value: never): never {  
    throw new Error(`Unhandled: ${JSON.stringify(value)}`);  
}  
  
function area(s: Shape): number {  
    switch (s.kind) {  
        case "circle": return Math.PI * s.radius ** 2;  
        case "square": return s.side ** 2;  
        case "rect":    return s.width * s.height;  
        default:        return assertNever(s); // s is `never` here  
    }  
}
```

- Add a fourth member and forget its `case` → **this line stops compiling**
- The compiler finds the missing branch for you, before runtime

User-Defined Type Guards

- A function whose return type is `x is Type` narrows at the **call site**

```
type Cat = { kind: "cat"; meow(): void };
type Dog = { kind: "dog"; bark(): void };

function isCat(pet: Cat | Dog): pet is Cat {
    return pet.kind === "cat";
}

const pets: (Cat | Dog)[] = loadPets();

const cats = pets.filter(isCat); // Cat[] - narrowed!
cats[0].meow();                  // safe
```

- A plain `boolean` return would leave `cats` as `(Cat | Dog)[]`

Switching on the Tag

Narrowing and Discriminated Unions

Give every member a shared tag, switch on it, and TypeScript knows exactly which shape you are holding.

```
type Shape =  
  | { kind: 'circle'; radius: number }  
  | { kind: 'square'; side: number }  
  | { kind: 'rect'; w: number; h: number };
```

kind is the DISCRIMINANT: a literal type unique to each member.

switch (s.kind)

case 'circle'
s is narrowed to the circle member.
s.radius is available; s.side is an error.

Why a tag beats a guess

Without a discriminant you end up probing for properties ('radius' in s), which breaks the moment two members share a field. A literal tag is unambiguous, cheap at run time, and survives JSON, so the same union works for data off the network. Narrowing is flow analysis: TypeScript tracks what must be true on each path. Reassigning the variable throws that away.

case 'square'
s.side is available, and nothing else.

case 'rect'
s.w and s.h are available.

default
const check: never = s;
All three handled, so s is never here; it compiles.
Add a fourth member and THIS LINE fails to compile, which is how you are told to update the switch.

Everything that narrows

typeof x === 'string'
for primitives
x instanceof Date
for class instances
'radius' in s
for plain object shapes
Array.isArray(x)
for arrays
if (x) / if (x !== null)
truthiness and null
x.kind === 'circle'
the discriminant: the one that scales

- Each branch narrows the union to exactly one member
- `never` in the default proves you covered every case